Assignment 3 Report

Name: Ruimin Zhang

UNI: rz2737

In [1]:
```
import os
import pickle

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("default")
sns.set_theme(style="whitegrid")

pd.set_option("display.max_columns", 50)
pd.set_option("display.width", 120)
```

1. Overview

The goal of this assignment is to use a Large Language Model (LLM) to extract structured financial information from unstructured news and then explore how this information relates to short-term stock price movements.

Concretely, the assignment consists of three main steps:

1. **Prompt construction (Task 1)**
   For each stock and week-long window, we build a prompt that includes a company introduction, the start and end prices, and the list of related news.
2. **LLM generation (Task 2)**
   We use a fine-tuned Llama3-LoRA model (FinGPT-style) to generate structured descriptions for each prompt.
3. **Parsing and analysis (Task 3)**
   We parse the LLM output into a structured dataframe, engineer simple numerical factors (e.g., price change and news count), and visualize the relationships between news and market returns.

2. Prompt Construction

In the first step, we construct a text prompt for each `(ticker, time window)` pair. The prompt contains:

- A **company introduction** (based on metadata such as sector, market cap, and primary exchange).

- A **price movement description** for a 1-week window, including:
  - Start date and end date
  - Start price and end price
- A list of **news headlines** and short summaries during that window.

A typical prompt looks like this:

> [Company Introduction]
> AbbVie Inc is a leading entity in the Biotechnology sector...
> From 2025-08-11 to 2025-08-18, ABBV's stock price increased from 197.22 to 205.09.
> News during this period are listed below:
>
> - Headline 1: ...
> - Headline 2: ...

Below, we show a simplified example of how such prompts are constructed from raw metadata and news.

In [3]:
```python
# Example: pseudo-code for building prompts from a dataframe
# (In the actual project, this logic lives in helper scripts such as `prompt.py`

def build_prompt_for_row(row):
    """
    Construct a single prompt for a given (ticker, window) row.
    """
    company_intro = row.get("company_intro", "")
    ticker = row["ticker"]
    start_date = row["start_date"]
    end_date = row["end_date"]
    start_price = row["price_from"]
    end_price = row["price_to"]

    # Here we assume `row["news"]` is a list of dicts with "headline" and "summa
    news_list = row.get("news", [])

    head = (
        "[Company Introduction]:\n"
        f"{company_intro}\n\n"
        f"From {start_date} to {end_date}, {ticker}'s stock price moved "
        f"from {start_price:.2f} to {end_price:.2f}. "
        "News during this period are listed below:\n\n"
    )

    news_lines = []
    for item in news_list:
        h = item.get("headline", "").strip()
        s = item.get("summary", "").strip()
        if h:
            if s:
                news_lines.append(f"- {h}  (Summary: {s})")
            else:
                news_lines.append(f"- {h}")

    prompt = head + "\n".join(news_lines)
    return prompt
```

```
# In practice, prompts are built for all rows and then passed to the LLM.
# Here we only illustrate the format.
```

3. LLM Generation

Once we have constructed the prompts, we feed them into a fine-tuned Llama3-LoRA model.

- **Base model:** `meta-llama/Meta-Llama-3-8B-Instruct`
- **Fine-tuning method:** LoRA (parameter-efficient)
- **Input:** the prompt shown in Section 2
- **Output:** a structured, human-readable description that includes:
  - A refined company introduction
  - Several time periods (weekly windows) with:
    - start date, end date
    - start price, end price
    - a list of related news items

The heavy LLM inference step is done once and the raw outputs are saved to disk for later analysis. In this report notebook, we focus on **how the outputs are parsed and analyzed**, rather than re-running the expensive generation.

4. Parsing LLM Output and Building a Dataframe

The LLM outputs are free-form text but follow a regular structure, for example:

- A **company introduction** block
- Several **period** blocks, each describing:
  - ticker
  - start date, end date
  - start price, end price
  - a list of news items

We implement a parser that:

1. Splits the text into introduction and period sections.
2. Extracts key fields (ticker, dates, prices) using simple string rules.
3. Collects news items into a list.
4. Converts all periods into a unified dataframe with the following columns:

- `company_intro`
- `ticker`
- `start_date`, `end_date`
- `price_from`, `price_to`
- `pct_change`
- `num_news`

For the analysis below, we directly load the parsed dataframe from disk.

In [6]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

In [7]:
```
parsed_path = "/content/drive/MyDrive/FinGPT_project/inference/hw3_parsed_llama3

with open(parsed_path, "rb") as f:
    df = pickle.load(f)

print("Dataframe shape:", df.shape)
df.head()
```

```
Dataframe shape: (30, 8)
```

Out[7]:

	company_intro	ticker	start_date	end_date	price_from	price_to	pct_change	r
0	AbbVie Inc is a leading entity in the Biotechn...	ABBV	2025-08-11	2025-08-18	197.22	205.09	0.039905	
1	AbbVie Inc is a leading entity in the Biotechn...	ABBV	2025-08-18	2025-08-25	205.09	206.06	0.004730	
2	AbbVie Inc is a leading entity in the Biotechn...	ABBV	2025-08-25	2025-08-29	206.06	208.89	0.013734	
3	Arch Capital Group Ltd is a leading entity in ...	ACGL	2025-08-11	2025-08-18	89.88	90.35	0.005229	
4	Arch Capital Group Ltd is a leading entity in ...	ACGL	2025-08-18	2025-08-25	90.35	91.88	0.016934	

In [8]:
```
# Make sure date columns are in datetime format
df["start_date"] = pd.to_datetime(df["start_date"])
df["end_date"] = pd.to_datetime(df["end_date"])

# Basic summary statistics
display(df.describe(include="all"))

print("\nTickers in the dataset:", sorted(df["ticker"].unique()))
```

	company_intro	ticker	start_date	end_date	price_from	price_to	pct_ch
count	30	30	30	30	30.000000	30.000000	30.00
unique	10	10	NaN	NaN	NaN	NaN	
top	AbbVie Inc is a leading entity in the Biotechn...	ABBV	NaN	NaN	NaN	NaN	
freq	3	3	NaN	NaN	NaN	NaN	
mean	NaN	NaN	2025-08-18 00:00:00	2025-08-24 00:00:00	167.132000	168.473667	0.00
min	NaN	NaN	2025-08-11 00:00:00	2025-08-18 00:00:00	70.610000	75.280000	-0.11
25%	NaN	NaN	2025-08-11 00:00:00	2025-08-18 00:00:00	90.732500	91.617500	-0.00
50%	NaN	NaN	2025-08-18 00:00:00	2025-08-25 00:00:00	165.825000	164.970000	0.00
75%	NaN	NaN	2025-08-25 00:00:00	2025-08-29 00:00:00	205.105000	205.822500	0.02
max	NaN	NaN	2025-08-25 00:00:00	2025-08-29 00:00:00	289.650000	314.700000	0.10
std	NaN	NaN	NaN	NaN	76.061571	77.655041	0.03

```
Tickers in the dataset: ['ABBV', 'ACGL', 'ADSK', 'AEP', 'AKAM', 'ALB', 'ALL', 'AL
LE', 'AMAT', 'AMGN']
```

5. Exploratory Analysis and Visualization

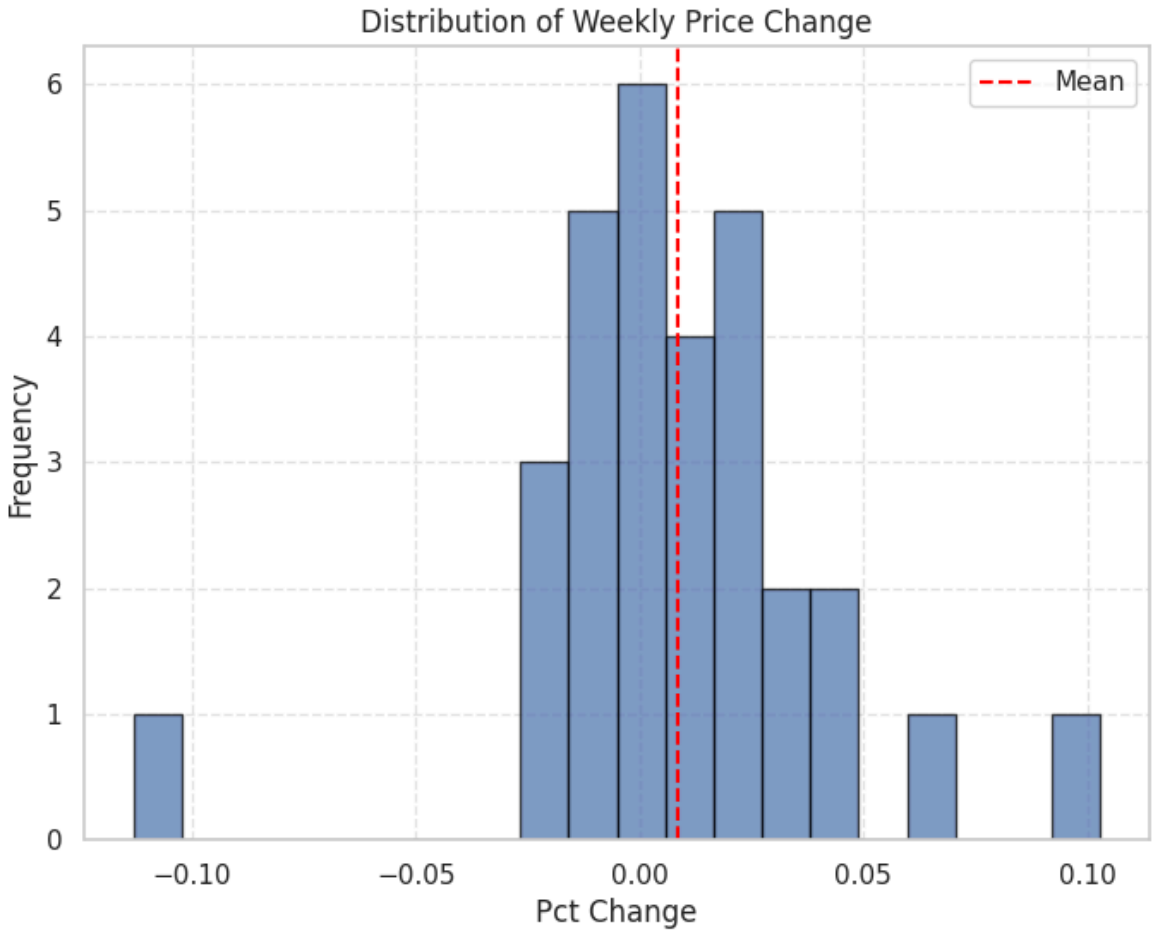
In this section, we use simple visualizations to understand:

1. The distribution of weekly price changes.
2. The distribution of the number of news articles per window.
3. The relationship between news volume and price changes.
4. Cross-sectional differences across tickers.

In [9]:
```
plt.figure(figsize=(8, 6))
plt.hist(df["pct_change"], bins=20, edgecolor="black", alpha=0.7)
plt.axvline(df["pct_change"].mean(), color="red", linestyle="--", label="Mean")

plt.title("Distribution of Weekly Price Change")
plt.xlabel("Pct Change")
plt.ylabel("Frequency")
plt.legend()
```

```
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



5.1 Distribution of Price Change

The histogram shows that most weekly price changes are close to zero, with a concentration between approximately -2% and +4%.

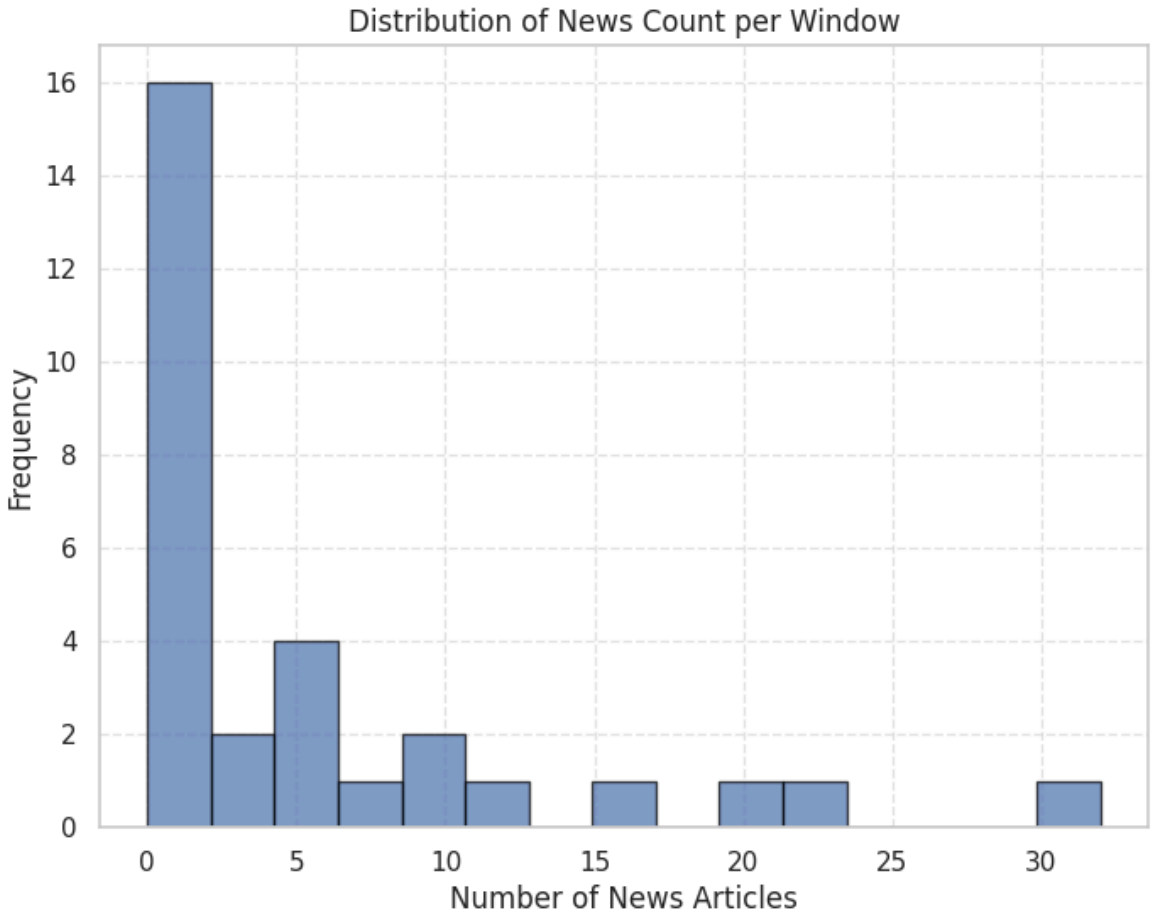
There are a few outliers:

- Some windows exhibit relatively large positive returns.
- At least one window shows a sizable negative return.

Overall, the distribution is reasonable for weekly stock returns and suggests that most weeks are relatively calm, with only a handful of extreme moves.

In [10]:
```
plt.figure(figsize=(8, 6))
plt.hist(df["num_news"], bins=15, edgecolor="black", alpha=0.7)

plt.title("Distribution of News Count per Window")
plt.xlabel("Number of News Articles")
plt.ylabel("Frequency")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



5.2 Distribution of News Count

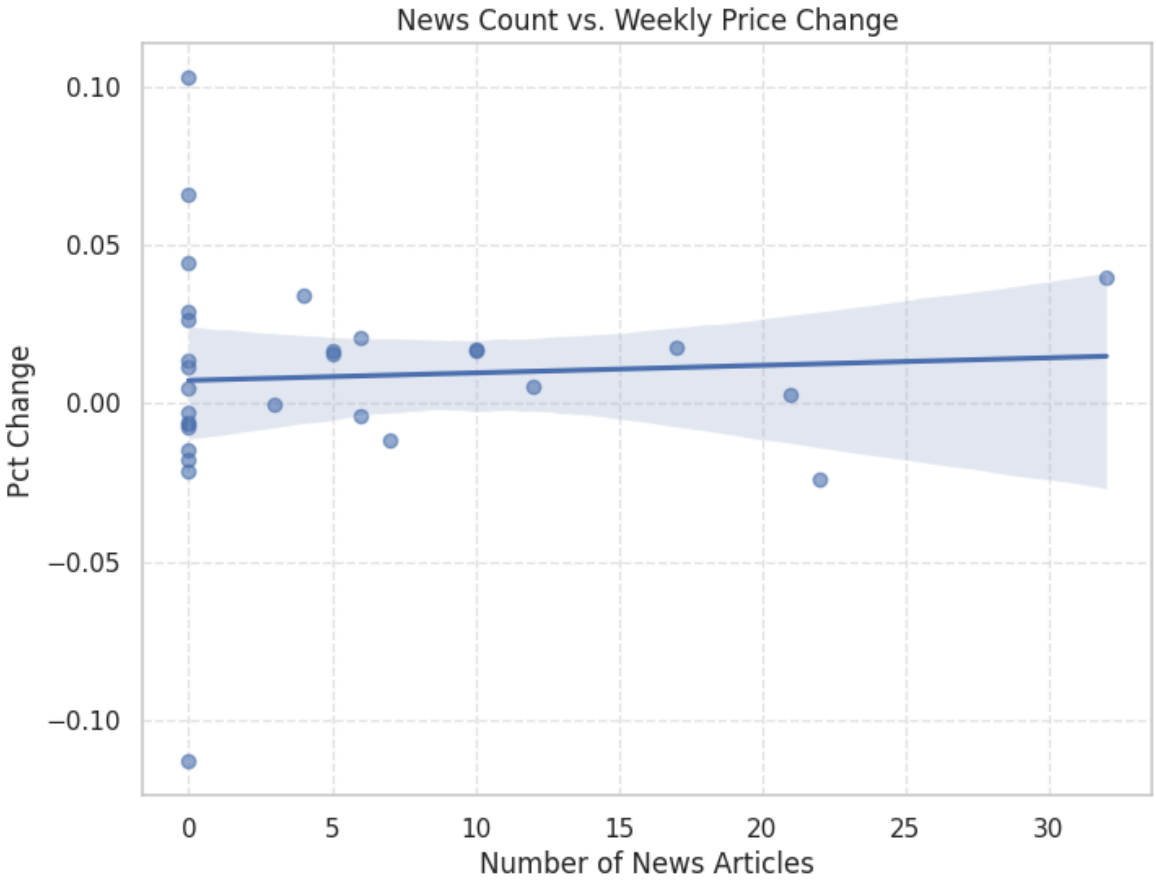
The news count is highly skewed:

- Many windows have only a few news articles.
- A small number of windows have a very large number of news items.

This heavy-tailed behavior is intuitive: most companies receive modest news coverage, while certain events (e.g., major earnings releases or FDA decisions) attract intense media attention in a short period.

In [11]:
```
plt.figure(figsize=(8, 6))
sns.regplot(
    data=df,
    x="num_news",
    y="pct_change",
    scatter_kws={"alpha": 0.6}
)

plt.title("News Count vs. Weekly Price Change")
plt.xlabel("Number of News Articles")
plt.ylabel("Pct Change")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



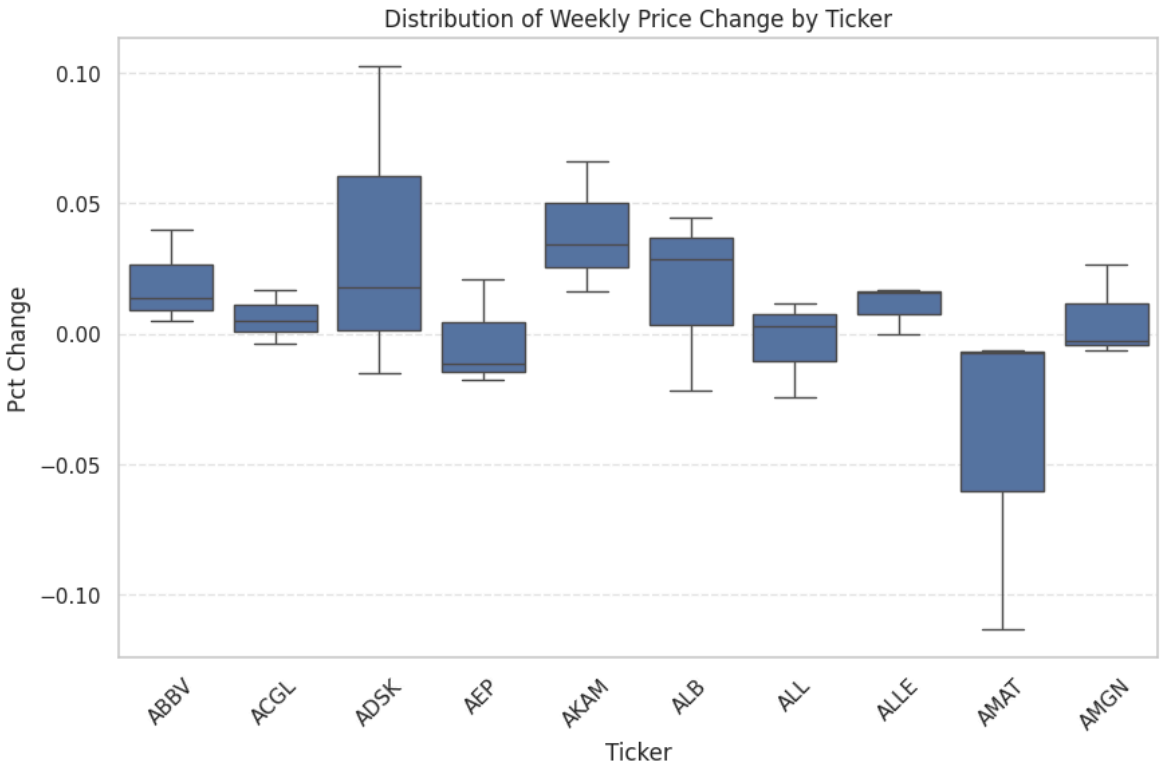
5.3 Relationship Between News Count and Price Change

The scatter plot with a fitted regression line suggests that:

- There is **no strong linear relationship** between the number of news articles and weekly price changes in this small sample.
- The regression slope is close to zero and the points are widely scattered.

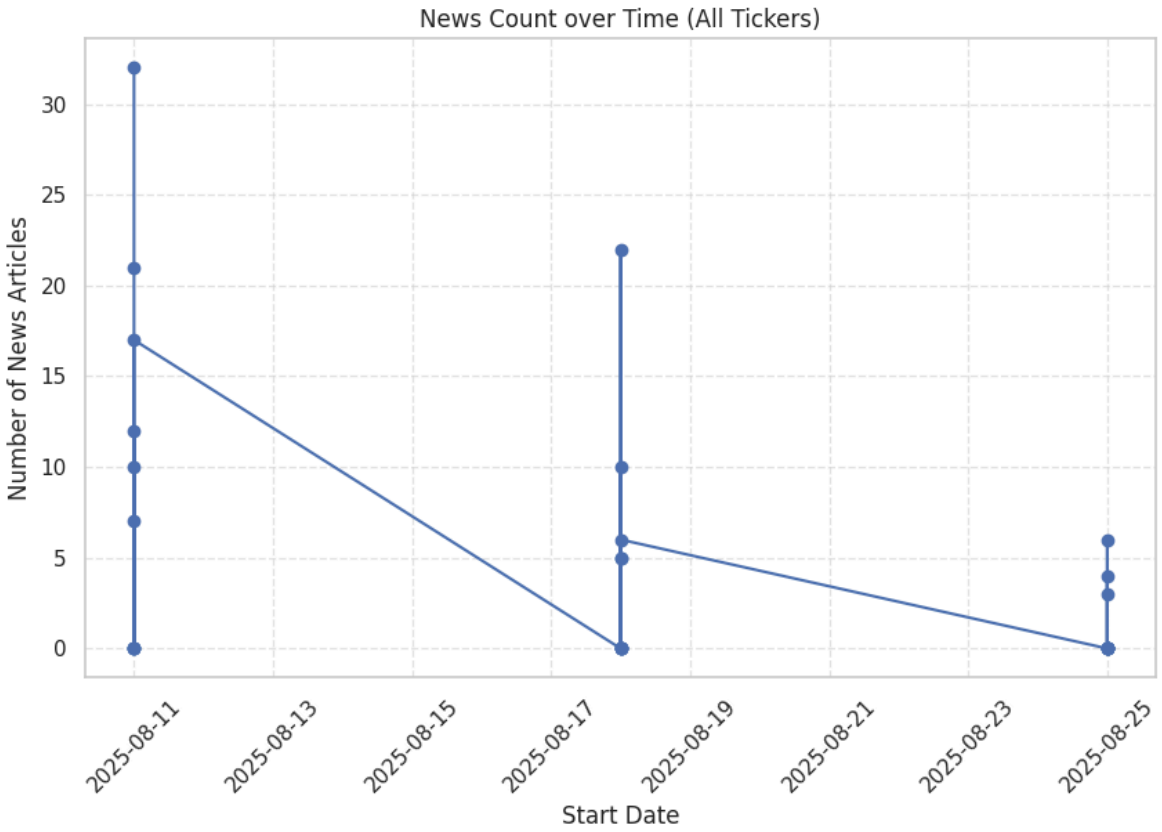
This implies that **news volume alone is not a strong predictor** of short-term stock returns. To extract more informative signals, one would likely need to incorporate *news sentiment*, *topic relevance*, or other qualitative features instead of just counting headlines.

In [12]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="ticker", y="pct_change")
plt.title("Distribution of Weekly Price Change by Ticker")
plt.xticks(rotation=45)
plt.xlabel("Ticker")
plt.ylabel("Pct Change")
plt.grid(True, axis="y", linestyle="--", alpha=0.5)
plt.show()
```



In [13]:
```
df_sorted = df.sort_values("start_date")

plt.figure(figsize=(10, 6))
plt.plot(df_sorted["start_date"], df_sorted["num_news"], marker="o")
plt.title("News Count over Time (All Tickers)")
plt.xlabel("Start Date")
plt.ylabel("Number of News Articles")
plt.xticks(rotation=45)
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



6. Discussion and Conclusion

This assignment demonstrates an end-to-end pipeline where a Large Language Model is used to turn unstructured news into structured, analyzable data:

1. **Prompt design**
   For each `(ticker, week)` window, we construct a prompt that combines company metadata, price movements, and a list of news headlines.

2. **LLM-based extraction**
   A fine-tuned Llama3-LoRA model produces structured descriptions, which are then parsed into a dataframe containing:

   - `ticker`, `start_date`, `end_date`
   - `price_from`, `price_to`, `pct_change`
   - `num_news` (number of news items)

3. **Exploratory analysis**
   Using this dataframe, we explore:

   - The distributions of weekly returns and news counts.
   - The cross-sectional behavior of different tickers.
   - The relationship between news volume and price changes.

Key observations:

- Weekly price changes are mostly small, with a few large positive or negative moves, which is realistic for equity markets.
- News coverage is highly uneven: most windows have little news, while a few windows are saturated with headlines.
- There is no strong linear relationship between **news volume** and **weekly returns** in this small sample.

Overall, this suggests that **LLM-based extraction is a powerful tool for structuring financial text**, but naive features such as raw news counts may not be sufficient to build profitable trading signals. Future work could incorporate:

- Sentiment scores from the LLM,
- Topic classification (e.g., earnings, M&A, regulation),
- Interactions between news and fundamental or technical factors.